



IMPLEMENTAÇÃO PERCEPTRON

APRENDIZAGEM DE MÁQUINA

CST em Desenvolvimento de Software Multiplataforma



PROF. Me. TIAGO A. SILVA



IMPLEMENTAÇÃO DO CÓDIGO

*Implementando árvores de decisão com
Python*

CÉLULA 1: IMPORTANDO AS BIBLIOTECAS

 Aula - Perceptron - Perceptron.ipynb

```
1  # Inclusão das bibliotecas usadas no trabalho
2  import numpy as np
3  import matplotlib.pyplot as plt
```

CARREGAMENTO DOS DADOS

Carregando o dataset

CÉLULA 2: CARREGANDO OS DADOS DO DATASET



Aula 7 - Árvore de Decisão - arvore_decisao.ipynb

```
2  # Função para carregar dados do CSV
3  def carregar_dados(arquivo):
4      X, y = [], []
5      with open(arquivo, 'r') as f:
6          leitor = csv.reader(f)
7          next(leitor) # pular cabeçalho
8          for linha in leitor:
9              X.append([float(linha[0]), float(linha[1])]) # idade, salario
10             y.append(int(linha[2]))
11     return np.array(X), np.array(y)
12
13 # Carregando o dataset
14 X, y = carregar_dados("dataset.csv")
15 print("Formato dos dados:", X.shape, y.shape)
16 print("Primeiras amostras:\n", X[:5], "\nRótulos:", y[:5])
```

CÉLULA 2: DECLARAÇÃO DA CLASSE



Aula - Perceptron - Perceptron.ipynb

```
1 class Perceptron:
2     def __init__(self, learning_rate=0.01, n_epochs=1000, weight_init="random"):
3         self.learning_rate = learning_rate
4         self.n_epochs = n_epochs
5         self.weights = None
6         self.weight_init = weight_init
7         self.bias = None
8         self.errors_per_epoch = [] # Armazena o erro a cada época
9
```

CÉLULA 2: INICIALIZAÇÃO DOS PESOS



Aula - Perceptron - Perceptron.ipynb

```
10 # Inicializando os pesos de forma customizada
11 def _initialize_weights(self, n_features):
12     """ Inicializa os pesos de acordo com o método escolhido """
13     if self.weight_init == "random":
14         self.weights = np.random.rand(n_features)
15         self.bias = np.random.rand()
16     elif self.weight_init == "zeros":
17         self.weights = np.zeros(n_features)
18         self.bias = 0
19     elif self.weight_init == "normal":
20         self.weights = np.random.randn(n_features) * 0.01 # Pequenos valores gaussianos
21         self.bias = np.random.randn() * 0.01
22     else:
23         raise ValueError("Opção inválida para inicialização dos pesos")
24
```



```
25 def fit(self, X, y):
26     n_samples, n_features = X.shape
27     #self.weights = np.zeros(n_features)
28     self._initialize_weights(n_features) # Inicializa os pesos dinamicamente
29     self.bias = 0
30
31     for epoch in range(self.n_epochs):
32         errors = 0 # Contador de erros por época
33
34         for i in range(n_samples):
35             linear_output = np.dot(X[i], self.weights) + self.bias
36             y_pred = np.sign(linear_output)
37
38             # Atualização dos pesos se houver erro
39             if y_pred != y[i]:
40                 self.weights += self.learning_rate * y[i] * X[i]
41                 self.bias += self.learning_rate * y[i]
42                 errors += 1 # Incrementa erro se classificação errada
43
44             # Armazena a taxa de erro (erros / total de amostras)
45             self.errors_per_epoch.append(errors / n_samples)
46
47             # Parada antecipada se não houver erros
48             if errors == 0:
49                 break
50
51     # Após o treinamento, calcular e exibir estatísticas
52     self._print_training_summary(X, y, epoch + 1)
53
```

CÉLULA 2: DECLARAÇÃO DO MÉTODO FIT

CÉLULA 2: DECLARAÇÃO DO MÉTODO PARA VER O RESULTADO DO TREINAMENTO



Aula - Perceptron - Perceptron.ipynb

```
54 # Método para exibir a acurácia e pesos finais
55 def _print_training_summary(self, X, y, epochs):
56     y_pred = self.predict(X)
57     accuracy = np.mean(y_pred == y) * 100
58     std_dev = np.std(self.weights)
59
60     print("\n==== Resumo do Treinamento =====")
61     print(f"Acurácia no conjunto de treino: {accuracy:.2f}%")
62     print(f"Número total de épocas: {epochs}")
63     print(f"Pesos finais aprendidos: {self.weights}")
64     print(f"Bias final: {self.bias}")
65     print("=====\n")
```

```
67 # Retorna a predição com base no treinamento do Perceptron
68 def predict(self, X):
69     return np.sign(np.dot(X, self.weights) + self.bias)
70
71 # Calculando as métricas da predição
72 def predict_summary(self, X, y, epochs, phase):
73     """ Método para exibir a acurácia, desvio padrão e pesos finais """
74     y_pred = np.sign(np.dot(X, self.weights) + self.bias)
75     accuracy = np.mean(y_pred == y) * 100
76     std_dev = np.std(self.weights)
77
78     print(f"\n==== Resumo do {phase} ====")
79     print(f"Acurácia no conjunto de {phase.lower()}: {accuracy:.2f}%")
80     print(f"Desvio Padrão dos Pesos: {std_dev:.5f}")
81     if epochs > 0:
82         print(f"Número total de épocas: {epochs}")
83     print(f"Pesos finais aprendidos: {self.weights}")
84     print(f"Bias final: {self.bias}")
85     print("=====\n")
```

CÉLULA 2: DECLARAÇÃO DO MÉTODO PREDICT

CÉLULA 2: MÉTODO PARA PLOTAR A CURVA DE ERROS



Aula - Perceptron - Perceptron.ipynb

```
87 # Plota o gráfico da evolução dos erros ao longo das épocas de treinamento.
88 def plot_error(self):
89     plt.figure(figsize=(8, 4))
90     plt.plot(self.errors_per_epoch, marker='o', linestyle='-')
91     plt.xlabel("Época")
92     plt.ylabel("Taxa de erro")
93     plt.title("Evolução do erro ao longo das épocas")
94     plt.grid()
95     plt.show()
```

CÉLULA 3: FUNÇÃO PARA PLOTAR A FRONTEIRA DE DECISÃO



Aula - Perceptron - Perceptron.ipynb

```
1  # Gráfico da "fronteira" gerado pelo Perceptron
2  def plot_decision_boundary(X, y, model, title="Fronteira de Decisão do Perceptron"):
3      x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
4      y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
5      xx, yy = np.meshgrid(np.linspace(x_min, x_max, 100), np.linspace(y_min, y_max, 100))
6
7      # Predição para cada ponto da grade
8      Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
9      Z = Z.reshape(xx.shape)
10
11     # Criar o gráfico
12     plt.contourf(xx, yy, Z, alpha=0.3)
13     plt.scatter(X[:, 0], X[:, 1], c=y, cmap="bwr", edgecolors="k")
14     plt.xlabel("X1")
15     plt.ylabel("X2")
16     plt.title(title)
17     plt.show()
```

```
1 # Função para plotar lado a lado: dataset e fronteira de decisão
2 def plot_comparison(X, y, model):
3     # Criar uma grade para o plano cartesiano
4     x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1
5     y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1
6     xx, yy = np.meshgrid(np.linspace(x_min, x_max, 100), np.linspace(y_min, y_max, 100))
7
8     # Predição para cada ponto da grade
9     Z = model.predict(np.c_[xx.ravel(), yy.ravel()])
10    Z = Z.reshape(xx.shape)
11
12    # Criar os subplots (1 linha, 2 colunas)
13    fig, axes = plt.subplots(1, 2, figsize=(12, 5))
14
15    # Subplot 1 - Fronteira de Decisão
16    axes[0].contourf(xx, yy, Z, alpha=0.3)
17    axes[0].scatter(X[:, 0], X[:, 1], c=y, cmap="bwr", edgecolors="k")
18    axes[0].set_xlabel("X1")
19    axes[0].set_ylabel("X2")
20    axes[0].set_title("Fronteira de Decisão do Perceptron")
21
22    # Subplot 2 - Apenas os Dados
23    axes[1].scatter(X[:, 0], X[:, 1], c=y, cmap="bwr", edgecolors="k")
24    axes[1].set_xlabel("X1")
25    axes[1].set_ylabel("X2")
26    axes[1].set_title("Distribuição dos Dados do Dataset")
27
28    plt.tight_layout()
29    plt.show()
```

CÉLULA 4: DECLARAÇÃO DO MÉTODO PLOT_COMPARISON

OBRIGADO!

- Encontre este **material on-line** em:
 - Slides: Google Classroom
- Em caso de **dúvidas**, entre em contato:
 - **Prof. Tiago:** tiago.silva@cps.sp.gov.br

